

SecureChat - Assignment #2

Information Security (Fall 2025)

Technical Documentation & Software Manual

Name: *Tauha Imran*

Roll No: *22i-1239*

Section: *Your Roll No*

Repository: <https://github.com/tauhaImran/securechat-A2>

TABLE OF CONTENTS

SecureChat - Assignment #2	
Information Security (Fall 2025)	
Technical Documentation & Software Manual	1
1.Assignment Overview	2
1.1 System Architecture	2
1.2 Features Implemented	2
Repository Structure	3
2.Software Implementation Tables	4
2.1. app/client.py	4
2.2. app/server.py	4
2.3 app/crypto	4
2.3.1 crypto/aes.py	4
AES-128-ECB Helpers with PKCS#7:	4
2.3.2 crypto/dh.py	5
2.3.3 crypto/pki.py	5
Functions	5
2.3.4 crypto/sign.py	6
Functions	6
2.4 app/common	7
2.4.1 common/protocol.py	7
Pydantic Models	7
2.4.2 common/utils.py	7
Helper Functions:	7
2.5. scripts/	8
2.5.1 scripts/gen_ca.py	8

Root CA Creation:	8
2.5.2 scripts/gen_cert.py	8
Issue Server/Client Certificate Signed by Root CA:	8
3. Evidence	8
3.1 Storage and Database tests	8
3.2 Full application Tests	10
3.3 Cryptography Tests	13
3.4 Scripts and Certificates tests	15
3.5 Common Functionality Tests	16

1. Assignment Overview

SecureChat is a secure end-to-end communication system designed to demonstrate key Information Security principles including **confidentiality**, **authentication**, **integrity**, and **non-repudiation**.

The project implements a full client–server secure messaging workflow using modern cryptographic primitives such as AES, Diffie–Hellman, RSA-based PKI, digital signatures, and certificate validation.

This assignment focuses on building:

- A **secure handshake protocol** (PKI + DH key exchange)
- An **encrypted messaging channel** (AES-128-ECB with PKCS#7 padding)
- A **certificate-based trust system** using a custom Root CA
- **Secure storage and transcript logging**
- Complete **testing** of all security components

The end result is a working secure chat application that enforces strong cryptographic guarantees and showcases security best-practices.

1.1 System Architecture

SecureChat follows a modular, layered architecture where each security component is isolated and testable.

The overall workflow is:

1. Client Initiation

- Client loads its certificate and private key
- Sends a **Hello** message containing its certificate
- The server validates it using the Root CA

2. Server Authentication

- Server returns its own certificate (ServerHello)
- Client validates the server identity and certificate chain

3. Diffie–Hellman Key Exchange

- Client sends DH parameters (p, g, A)
- Server responds with its DH public value (B)
- Both derive a shared AES-128 key using SHA-256 → 16 bytes

4. Encrypted Communication

- All messages are encrypted with AES-128-ECB + PKCS#7
- Chat messages use a structured JSON protocol
- Each message includes a digital signature for integrity and non-repudiation

5. Transcript Logging & Receipts

- Every encrypted message and receipt is stored in a transcript object

- Receipts include:
 - user identity
 - transcript hash
 - RSA signature

This allows both parties to later prove message delivery and content authenticity.

6. Storage Layer

- Simple database wrapper for storing:
 - Users
 - Password hashes
 - Message transcripts

This architecture ensures clean separation of:

Layer	Description
Crypto Layer	AES, DH, Signatures, PKI
Protocol Layer	Pydantic models, message formats
Network Layer	Client/server socket communication
Storage Layer	Transcript + database handling

Scripts

Certificate creation and debugging

1.2 Features Implemented

Security & Cryptography

- AES-128-ECB encryption with PKCS#7 padding
- Secure Diffie–Hellman key exchange (2048-bit MODP Prime)
- RSA-based PKI with custom Root CA
- Certificate verification:
 - signature validation
 - CA trust validation
 - hostname validation (CN / SAN)
 - validity period checks
- RSA digital signatures (SHA-256, PKCS#1 v1.5)

Messaging Protocol

- Authenticated handshake using certificates
- Encrypted and signed messages
- Delivery receipts with transcript hashing
- Pydantic-based structured protocol models

Server Functionality

- User registration and login (with hashed passwords)
- Secure session establishment
- Transcript logging per session
- User identity verification via certificate matching

Client Functionality

- Establish secure connections
- Validate server certificate
- Derive symmetric encryption keys
- Send encrypted signed messages
- Display receipts and transcript verification

Scripts & Utilities

- `gen_ca.py` — create self-signed Root CA
- `gen_cert.py` — issue server/client certificates
- Diagnostic script for PKI debugging
- Base64 helpers, hashing utilities, timestamping

Testing

- Encryption/decryption tests
- DH shared secret agreement tests
- PKI and certificate-validation tests

- Message signature tests
- Full client–server integration tests
- Database and transcript verification tests

Together, these features satisfy the core security principles:

Principle	Provided By
Confidentiality	AES encryption
Integrity	SHA-256 + signatures
Authentication	PKI certificates
Non-Repudiation	Digital signatures + transcripts

Quick Summary of Directories

- **app/** — main application code
 - **app/crypto/** — all cryptography implementations
 - **app/common/** — protocol + utilities
 - **app/storage/** — message/database storage
 - **scripts/** — certificate generation, CA creation
 - **certs/** — Root CA + server/client certificates
 - **transcripts/** — saved chat transcripts
 - **tests/** — functional and security tests
-

Repository Structure

```
securechat-A2/
├─ README.md
├─ requirements.txt
├─ .gitignore
├─ certs/
│   ├── MyRootCA_ca_cert.pem
│   ├── MyRootCA_ca_key.pem
│   ├── client.example.com_cert.pem
│   ├── client.example.com_key.pem
│   ├── myserver.example.com_cert.pem
│   └── myserver.example.com_key.pem
├─ transcripts/
│   └─ demo_session.json
├─ scripts/
│   ├── gen_ca.py
│   ├── gen_cert.py
│   ├── diag_pki.py
│   └── cli_commands.txt
├─ scripts_ss/                                # Support / dev scripts
├─ app/
│   ├── .env
│   ├── client.py
│   ├── server.py
│   ├── app_ss/                                # Reference / assignments
│   ├── common/
│   ├── protocol.py
│   ├── utils.py
│   ├── common_ss/
│   ├── crypto/
│   │   ├── aes.py
│   │   ├── dh.py
│   │   ├── pki.py
│   │   └── sign.py
│   ├── crypto_ss/
│   ├── storage/
│   │   ├── db.py
│   │   └── transcript.py
│   └── testing_files/
├─ tests/
│   └─ manual/
└─ NOTES.md
```

2. Software Implementation Tables

2.1. app/client.py

— add in

2.2. app/server.py

— add in

2.3 app/crypto

2.3.1 crypto/aes.py

AES-128-ECB Helpers with PKCS#7:

- `pkcs7_pad(data: bytes) -> bytes` — Pad input bytes to 16-byte block with PKCS#7
- `pkcs7_unpad(data: bytes) -> bytes` — Remove PKCS#7 padding; raises `ValueError` if invalid
- `encrypt_aes(key: bytes, plaintext: bytes) -> bytes` — AES-128-ECB encrypt with PKCS#7 padding
- `decrypt_aes(key: bytes, ciphertext: bytes) -> bytes` — AES-128-ECB decrypt and remove PKCS#7 padding

Notes:

- `key` must be exactly 16 bytes (AES-128).
- `encrypt_aes` outputs raw bytes (can be base64-encoded for JSON transport).
- `decrypt_aes` returns the original plaintext bytes.

2.3.2 crypto/dh.py

Constants:

- **P_2048** — 2048-bit MODP safe prime
- **G = 2** — generator

Functions:

- **generate_dh_pair()** -> **Tuple[dh.DHPrivateKey, dh.DHPublicKey]** — Generate fresh DH key pair
- **get_dh_public_bytes(public_key: dh.DHPublicKey)** -> **int** — Extract public value as integer
- **compute_shared_secret(own_private_key: dh.DHPrivateKey, peer_public_value: int)** -> **bytes** — Compute raw shared secret
- **derive_aes_key(shared_secret: bytes)** -> **bytes** — Truncate SHA-256 to 16 bytes for AES
- **client_dh_initiate()** -> **Tuple[dh.DHPrivateKey, dict]** — Client DH keypair + first message
- **server_dh_respond(client_msg: dict, server_private_key: dh.DHPrivateKey)** -> **Tuple[bytes, dict]** — Server DH response + AES key
- **client_dh_finalize(client_private_key: dh.DHPrivateKey, server_msg: dict)** -> **bytes** — Client computes AES key from server reply

2.3.3 crypto/pki.py

Functions

- **load_certificate(pem_path: str)** -> **Certificate** — Load X.509 certificate from PEM

- `load_ca_certificate(ca_pem_path: str) -> Certificate` — Load trusted Root CA cert
- `verify_signature(ca_cert: Certificate, leaf_cert: Certificate) -> bool` — Check leaf signed by CA
- `check_validity(cert: Certificate) -> bool` — Check cert validity window
- `get_common_name(cert: Certificate) -> Optional[str]` — Extract CN from subject
- `get_san_dns_names(cert: Certificate) -> List[str]` — Get DNS names from SAN
- `verify_identity(cert: Certificate, expected_hostname: str) -> bool` — Check CN/SAN matches hostname
- `is_ca_certificate(cert: Certificate) -> bool` — Check BasicConstraints ca=True
- `validate_certificate(leaf_pem_path: str, ca_pem_path: str, expected_hostname: str) -> Certificate` — Full cert validation (chain, CA, hostname)
- `validate_server_certificate(server_cert_pem: str, ca_pem_path: str, expected_server_name: str) -> Certificate` — Client-side server cert check
- `validate_client_certificate(client_cert_pem: str, ca_pem_path: str, expected_client_name: Optional[str] = None) -> Certificate` — Server-side client cert check

2.3.4 crypto/sign.py

Functions

- `sign_data(private_key_pem: str, data: bytes) -> bytes` — Sign data using RSA PKCS#1 v1.5 SHA-256

- `verify_signature(public_key_pem: str, data: bytes, signature: bytes) -> bool` — Verify RSA PKCS#1 v1.5 SHA-256 signature

2.4 app/common

2.4.1 common/protocol.py

Pydantic Models

- `BaseMessage(type: str, timestamp: int)` — Base message class for all messages
- `Hello(client_cert: str)` — Client hello / handshake message
- `ServerHello(server_cert: str)` — Server hello / handshake message
- `Register(username: str, password_hash: str)` — User registration message
- `Login(username: str, password_hash: str)` — User login message
- `DHClient(p: str, g: str, A: str)` — Client DH key exchange message
- `DHServer(B: str)` — Server DH key exchange message
- `Message(sender: str, ciphertext: str)` — Encrypted chat message
- `Receipt(identity: str, transcript_hash: str, signature: str)` — Message receipt / acknowledgement

2.4.2 common/utils.py

Helper Functions:

- `now_ms()` -> `int` — Current time in milliseconds

- `b64e(b: bytes) -> str` — Base64 encode bytes to string
- `b64d(s: str) -> bytes` — Base64 decode string to bytes
- `sha256_hex(data: bytes) -> str` — SHA-256 hash as hex string

2.5. scripts/

2.5.1 scripts/gen_ca.py

Root CA Creation:

- `create_root_ca(ca_name: str, output_dir: str = "certs") -> None` — Generate self-signed RSA Root CA

2.5.2 scripts/gen_cert.py

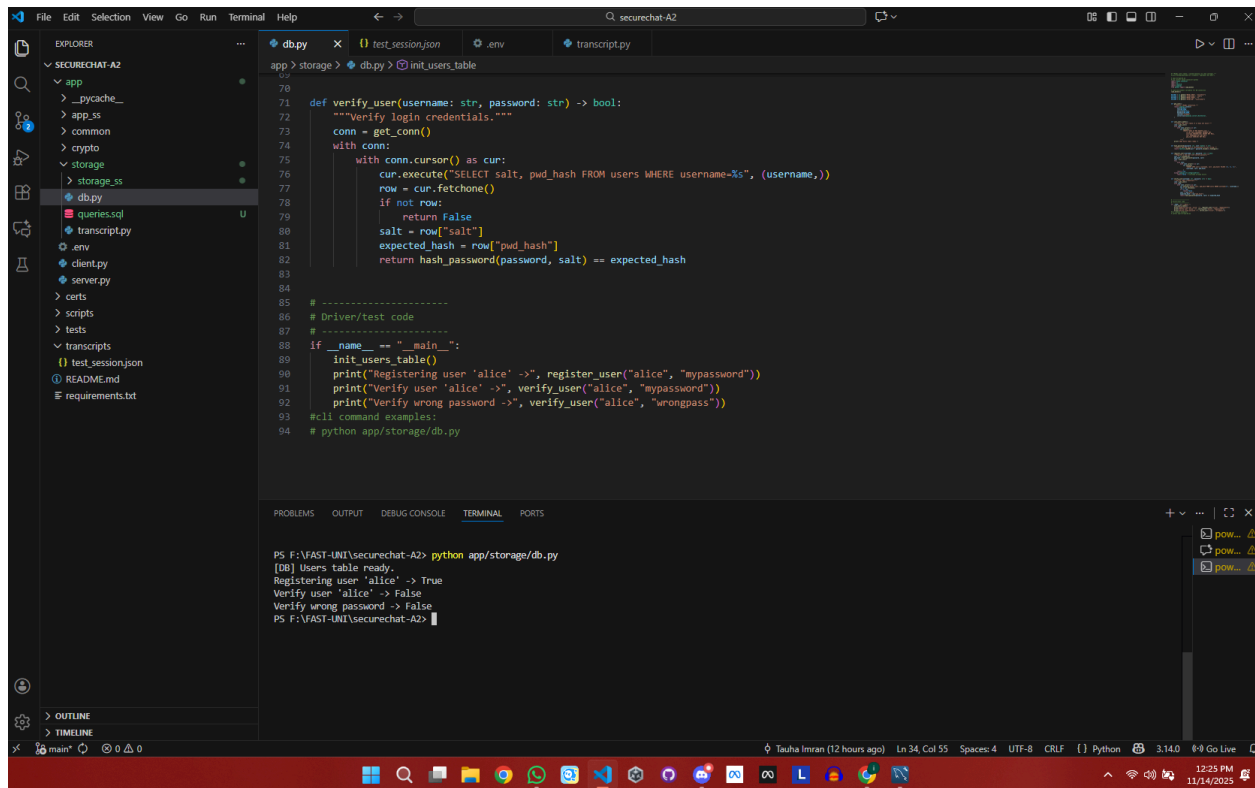
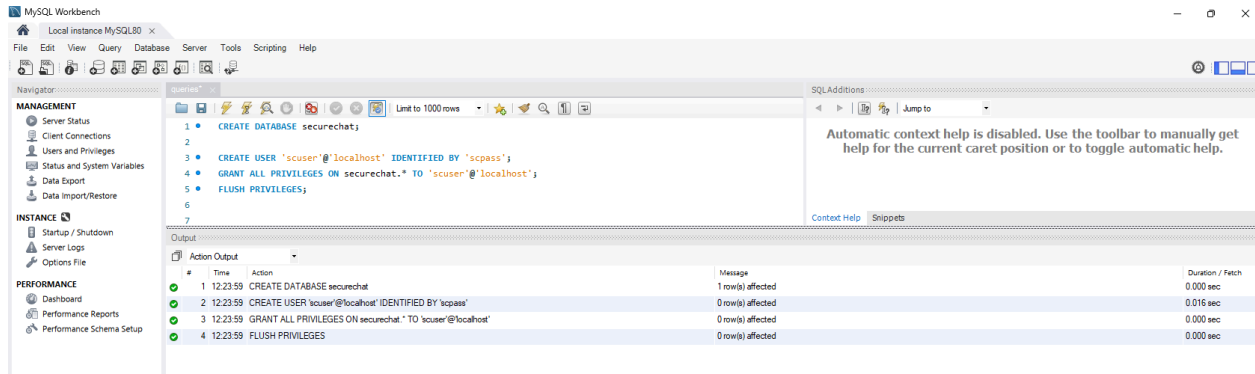
Issue Server/Client Certificate Signed by Root CA:

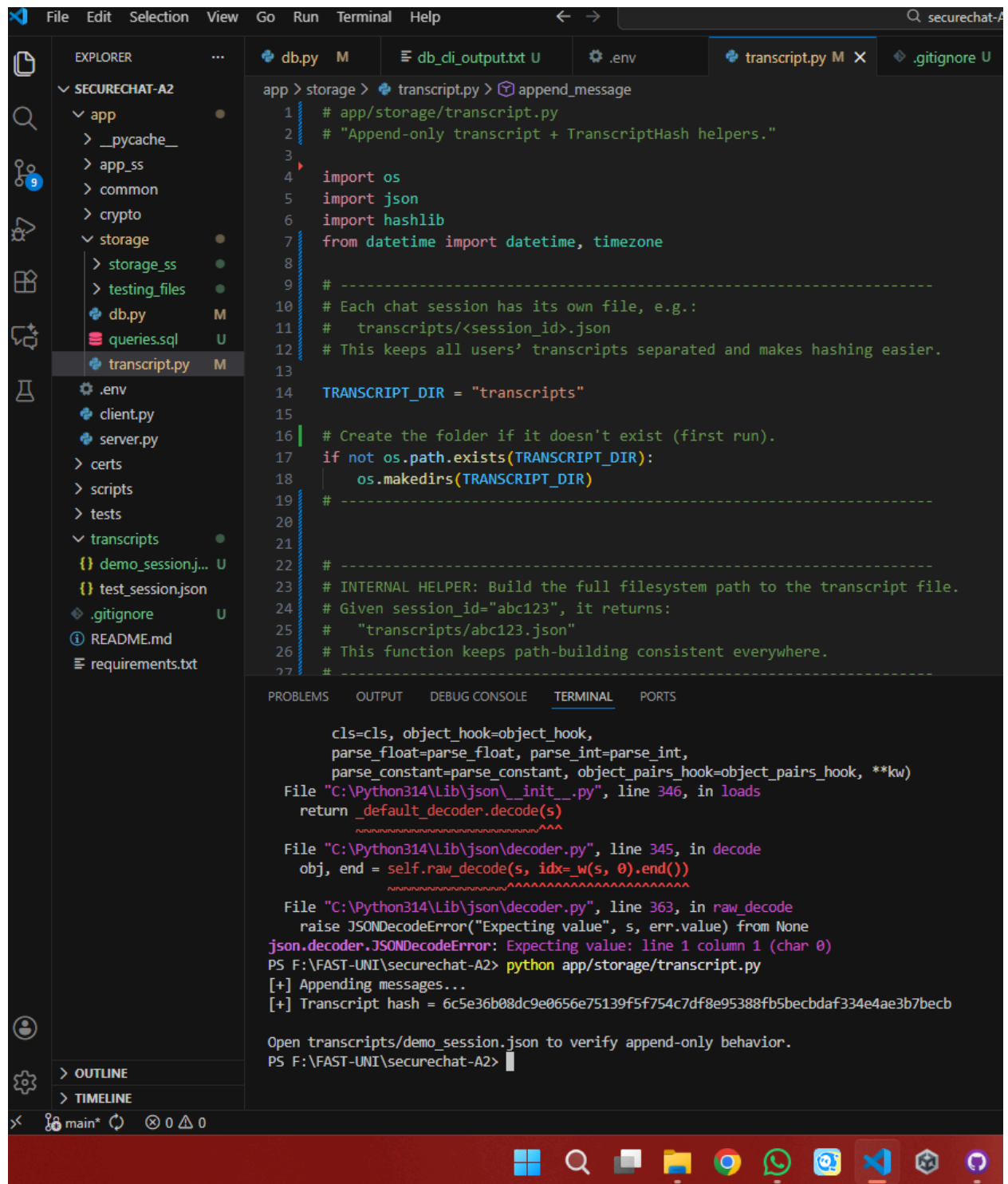
- `issue_certificate(ca_name: str, cert_name: str, cert_type: str, output_dir: str = "certs") -> None` — Generate server/client X.509 certificate

3. Evidence

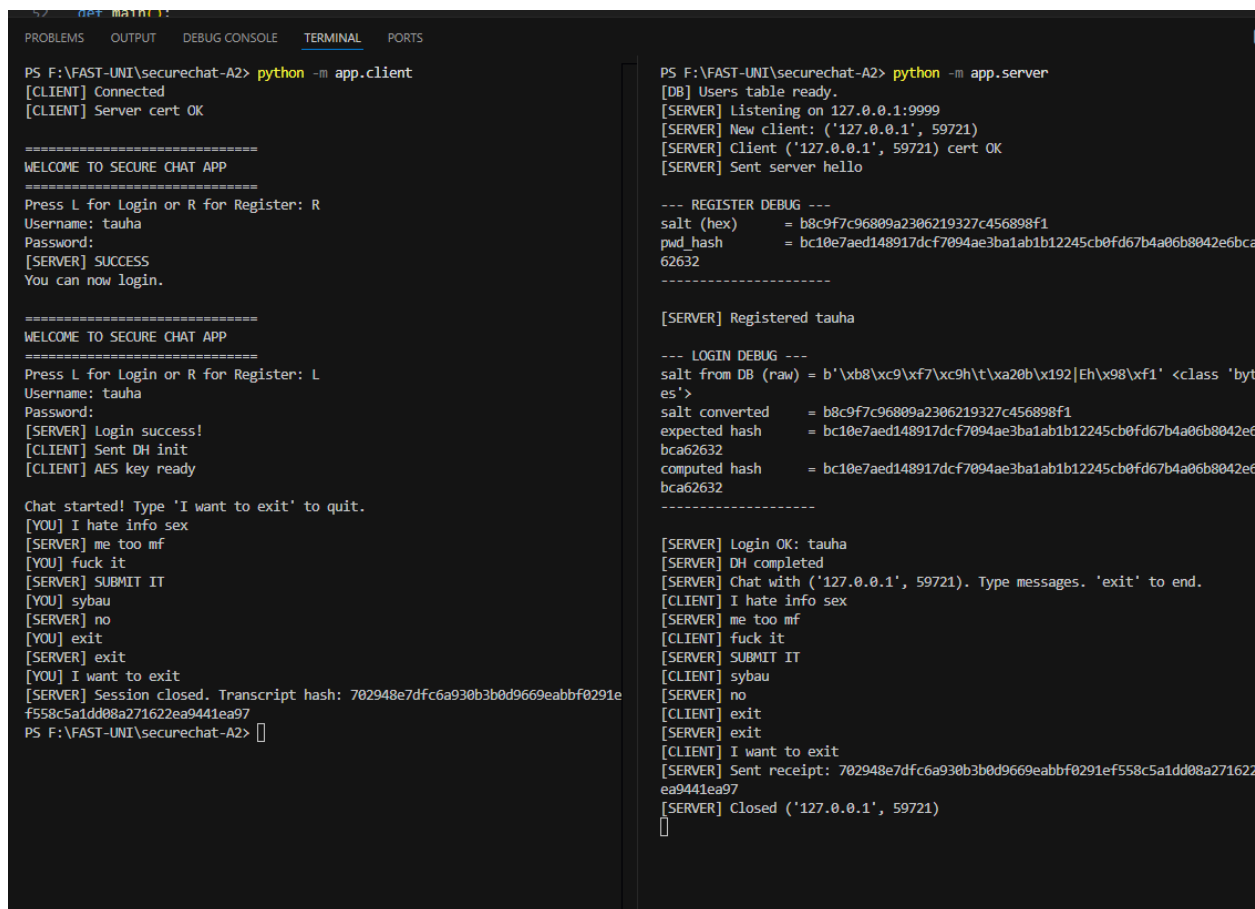
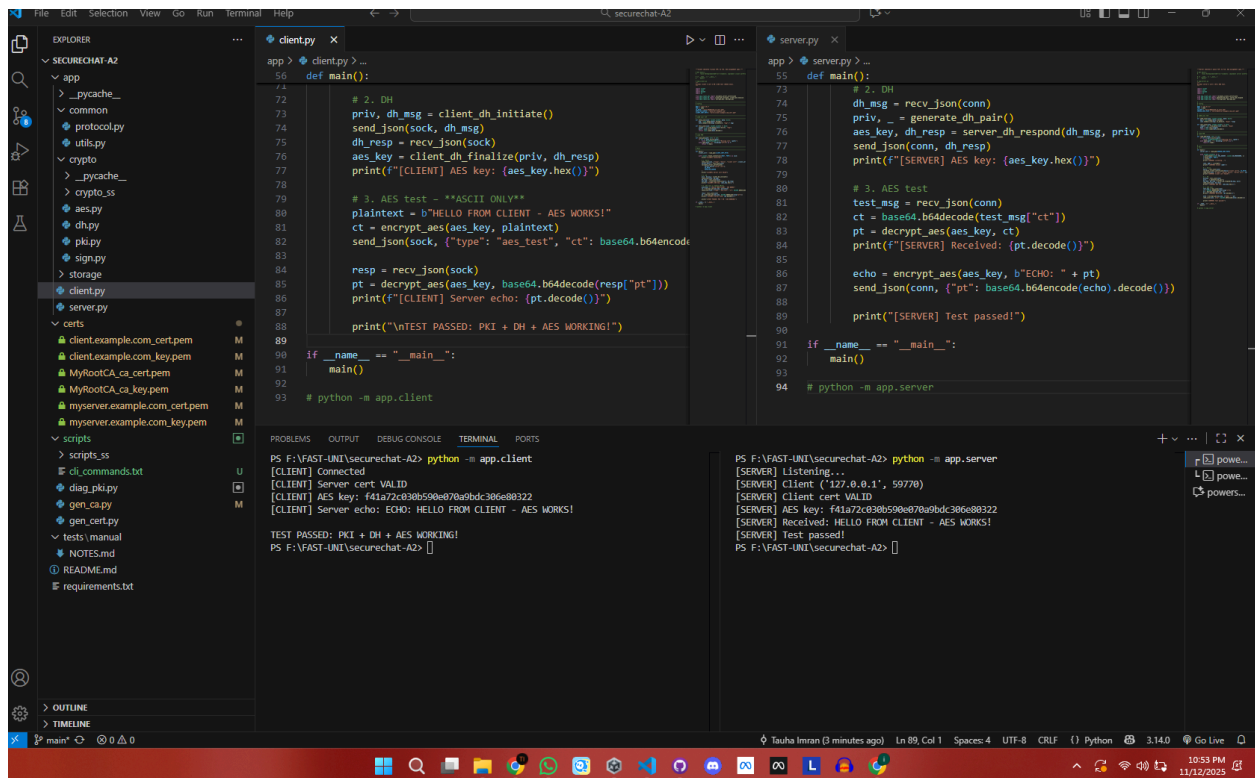
The following screenshots are tests of individual segments and collective ones too covering all aspects of confidentiality, Authenticity, Integrity and Non-Repudation

3.1 Storage and Database tests





3.2 Full application Tests



3.3 Cryptography Tests

```
app> crypto > aes.py ...
from app.crypto import client_on_initiate, client_on_finalize, generate_on_pair, server_on_respond

# Simulate DH to get shared key
client_priv, client_msg = client_on_initiate()
server_priv, _ = generate_on_pair()
server_key, server_resp = server_on_respond(client_msg, server_priv)
client_key = client_on_finalize(client_priv, server_resp)

assert server_key == client_key
key = client_key

# Test encryption/decryption
msg = b'Hello, Secure Chat!'
ct = encrypt_aes(key, msg)
pt = decrypt_aes(key, ct)

print(f'Original : {msg}')
print(f'Ciphertext (hex): {ct.hex()}')
print(f'Decrypted: {pt}')
assert pt == msg
print("AES-128-ECB + PKCS#7 test passed!")

python -m app.crypto.aes
```

20671792763873178289735836993426726239478442419554294384867284920978218380529595820722824465945755783025183075515171417049431753135923460476506247613221086916680693856669472756295730710856785431824
721286182123743396105619245320831189618159667400179523636890626183839677456243043931618199007421
Server = ('type': 'dh server', 'B': 187239227321268073796684340279672505067477080896724221594857493135068202128085254037306387456634692732351110003772454852634706548599789251065399357278532206318866
62731822146636821203158006944226141960231878510228376889237684914518413669364147789917661598414971373955181994513717246520619748900503535161891321688848391354376523916520486700267709061487710108
21013791974668257904185973408187837163850810982349924589509261086683393284112735190309527427425672158937691998463485257758101201747031894743697324648301484936417861773068322969739134150171950025
17896553029189986807154166043774369678859676173834733670
Shared AES-128 key derived: c33e21e1505166789eb5100che9516a
DH exchange successful!
PS F:\FAST-UNI\securechat-A2> python -m app.crypto.aes
Original : b'Hello, Secure Chat!'
Ciphertext (hex): c58860aef582721bf59fee98bdaef66c2299a7f64e9268b28b5233d2a5d72e
Decrypted: b'Hello, Secure Chat!'
AES-128-ECB + PKCS#7 test passed!
PS F:\FAST-UNI\securechat-A2>]

```
app> crypto > dh.py ...
def client_on_finalize():
    B = server_msg["B"]
    shared = compute_shared_secret(client_private_key, B)
    return derive_aes_key(shared)

# Test when run directly
if __name__ == "__main__":
    # Simulate full exchange
    client_priv, client_init = client_on_initiate()
    print("Client +", client_init)

    server_priv, _ = generate_on_pair()
    server_key, server_resp = server_on_respond(client_init, server_priv)
    print("Server +", server_resp)

    client_key = client_on_finalize(client_priv, server_resp)

    assert server_key == client_key, "DH key mismatch!"
    print("Shared AES-128 key derived:", client_key.hex())
    print("DH exchange successful!")

python -m app.crypto.dh
```

PS F:\FAST-UNI\securechat-A2> python -m app.crypto.dh
Client = ('type': 'dh client', 'g': 2, 'p': 3231700607131180738033891392642382824881794124114023911284208975140674170663435422261968941736356934711790173790970419175460587320919502885735898618562215
321275412514001774526278235796078236248884261084773870411895289406994117234542662521932384991981708953423551912507915870117001058587705103886184728025797085490350973250152616708133936179954
123647655916830317087220731703048908805671080077802194168647225078101123642919536104716365328971707744822790838535369288642566368772562895550929236375111740869729886841854355948669
32916421362182317879909994486524682624167203501185240745361090559, 'A': 2763820947698205173184368991438775383670523188198573471928181001415455994929968485979667268181136736774012192563701941463
55957522803052895642760665027612543461590518430419572394770437253969334029989021253113240883718158359847165891264444747628292365601186205136214539646959822360818752728043142461784876038970569936
20671792763873178289735836993426726239478442419554294384867284920978218380529595820722824465945755783025183075515171417049431753135923460476506247613221086916680693856669472756295730710856785431824
721286182123743396105619245320831189618159667400179523636890626183839677456243043931618199007421
Server = ('type': 'dh server', 'B': 187239227321268073796684340279672505067477080896724221594857493135068202128085254037306387456634692732351110003772454852634706548599789251065399357278532206318866
62731822146636821203158006944226141960231878510228376889237684914518413669364147789917661598414971373955181994513717246520619748900503535161891321688848391354376523916520486700267709061487710108
21013791974668257904185973408187837163850810982349924589509261086683393284112735190309527427425672158937691998463485257758101201747031894743697324648301484936417861773068322969739134150171950025
17896553029189986807154166043774369678859676173834733670
Shared AES-128 key derived: c33e21e1505166789eb5100che9516a
DH exchange successful!
PS F:\FAST-UNI\securechat-A2>]

```
File Edit Selection View Go Run Terminal Help
securechat-A2

EXPLORER
SECURECHAT-A2
  app
  common
  crypto
  _pycache_
  aes.py
  dh.py
  pk.py
  sign.py
  storage
  client.py
  server.py
  certs
    MyRootCA_ca_cert.pem
    MyRootCA_ca_key.pem
    myserver.example.com_cert.pem
    myserver.example.com_key.pem
  scripts
  scripts_ss
  gen_ca.py
  gen_cert.py
  tests
    manual
    NOTES.md
    README.md
    requirements.txt

pk.py
sign.py
client.py
server.py
certs
scripts
scripts_ss
gen_ca.py
gen_cert.py
tests
manual
NOTES.md
README.md
requirements.txt

def validate_client_certificate(
    cert = x509.load_pem_x509_certificate(client_cert_pem.encode("utf-8"), backend)

    if not verify_signature(ca_cert, cert):
        raise ValueError("Client certificate not signed by trusted CA")
    if not check_validity(cert):
        raise ValueError("Client certificate expired or not yet valid")
    if expected_client_name and not verify_identity(cert, expected_client_name):
        raise ValueError(f"Client identity mismatch: {expected_client_name}")
    raise ValueError(f"Client identity mismatch: {expected_client_name}")

    return cert

# Simple CLI test
if __name__ == "__main__":
    import sys
    if len(sys.argv) != 4:
        print(
            "Usage: python -m app.crypto.pki <leaf.pem> <ca.pem> <expected_hostname>"
        )
        sys.exit(1)

    leaf_path, ca_path, hostname = sys.argv[1], sys.argv[2], sys.argv[3]
    try:
        cert = validate_certificate(leaf_path, ca_path, hostname)
        print(f"Certificate VALID for {hostname}")
        print(f"Subject : {cert.subject}")
        print(f"Expires : {cert.not_valid_after_utc}")
    except Exception as exc:
        # pragma: no cover
        print(f"Validation FAILED: {exc}")

# to run this script: python -m app.crypto.pki certs/myserver.example.com_cert.pem certs/MyRootCA_ca_cert.pem myserver.example.com

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS F:\FAST-UNIT\securechat-A2> python -m app.crypto.pki certs/myserver.example.com_cert.pem certs/MyRootCA_ca_cert.pem myserver.example.com
PS F:\FAST-UNIT\securechat-A2> python -m app.crypto.pki certs/myserver.example.com_cert.pem certs/MyRootCA_ca_cert.pem myserver.example.com
Certificate VALID for myserver.example.com
Subject : <Name(<CN=,ST=ICT,L=Islamabad,O=SecureChat,OU=myserver.example.com>)>
Expires : 2028-02-13 15:03:57+00:00
PS F:\FAST-UNIT\securechat-A2>

main* 0 0 0
```

```
File Edit Selection View Go Run Terminal Help
securechat-A2

EXPLORER
SECURECHAT-A2
  app
  common
  crypto
  _pycache_
  aes.py
  dh.py
  pk.py
  sign.py
  storage
  client.py
  server.py
  certs
    client.example.com_cert.pem
    client.example.com_key.pem
    MyRootCA_ca_cert.pem
    MyRootCA_ca_key.pem
    myserver.example.com_cert.pem
    myserver.example.com_key.pem
  scripts
  scripts_ss
  cli_commands.txt
  diag_pk.py
  gen_ca.py
  gen_cert.py
  tests
    manual
    NOTES.md
    README.md
    requirements.txt

sign.py
storage
client.py
server.py
certs
scripts
scripts_ss
cli_commands.txt
diag_pk.py
gen_ca.py
gen_cert.py
tests
manual
NOTES.md
README.md
requirements.txt

# RSA PKCS#1 v1.5 SHA-256 sign/verify.
#raise NotImplementedError("students: implement RSA helpers")

# app/crypto/sign.py

# Import necessary modules from the cryptography library
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.backends import default_backend

#this function signs data using RSA PKCS#1 v1.5 with SHA-256
def sign_data(private_key_pem: str, data: bytes) -> bytes:
    # Sign the given data using RSA PKCS#1 v1.5 with SHA-256.
    # > param private_key_pem: The private key in PEM format.
    # > param data: The data to be signed.
    # > return: The signature as bytes.

    # load the private key from PEM format
    # accept PEM as text (str) or bytes; normalize to bytes for loading
    if isinstance(private_key_pem, str):
        pem_bytes = private_key_pem.encode("utf-8")
    else:
        pem_bytes = private_key_pem

    private_key = serialization.load_pem_private_key(
        pem_bytes, password=None, backend=default_backend()
    )

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS F:\FAST-UNIT\securechat-A2> ^C
PS F:\FAST-UNIT\securechat-A2> python app/crypto/sign.py
Signature: 00ce31974d578c3cf7df6967ae5df642e95e1a3482877a27e32eebf6314b5969c6c6e0b3675e21adbf2b3fbbf10bc352fca5a193e7f28b9c4746dd8e8ada4b2b3236cae87af7b4825781fa4ed6af4eac3112d6da95f51405fb311beac8d91203760a3c23f1b186cfc3c52f6c5b0d1b554d8d7e89f938b2efc70aee3554a9b0eb35158fe46688e5c81145e516f06746572be1b19456c52c3bdaebcb155f48a63a3e2d7f6a3c380e39967968a2a2f3e574b7cf3a328a9fb3095fd2a89299b3ae33c56ed1bf26ced8386c575953bb7fc3a82955579b9f2a7970a8f5211ec2eefb49570eb77c5873acc6cdd118614e3dd
Is the signature valid? True
PS F:\FAST-UNIT\securechat-A2> python app/crypto/sign.py
Signature: d4fe07af2b2510a99b031d1e37eb38f151b979446f1141d3fa8bcf4f3ac4887070faf9f975f6217796f807e17f33d5d21d18a249591f808c3be071b2030564b4f5c7d03019e0449b4c17443d375bfe79868c8a3e5a78cb444890c5468904c9cf18ac25ee9643821cb4d9a0d5963c29839b019034895f33c2325d8e967e9a349d81a5dea887492143d3112884c1e37864a296d9afaf46304bd35b7e04f735474211b005628be48974975830eb25beedecba6fb203b7075b3db0166523e27454016ee86f02b7e737ed520cf7f391d888b8edf61c893b595de5ef8ea3cda5a1eff076ae2bea8ff774af5dfcf3c620f3845d20676d5cf3fe8287727df998
Is the signature valid? True
PS F:\FAST-UNIT\securechat-A2>

main* 0 0 0
```

3.4 Scripts and Certificates tests

Explorer pane (left):

- SECURECHAT-A2
 - app
 - common
 - protocol.py
 - utils.py
 - crypto
 - aes.py
 - dh.py
 - pk.py
 - sign.py
 - storage
 - client.py
 - server.py
 - certs
 - MyRootCA_ca_cert.pem
 - MyRootCA_ca_key.pem
 - scripts
 - tests
 - README.md
 - requirements.txt

Terminal output:

```
certs > MyRootCA_ca_key.pem
1 -----BEGIN RSA PRIVATE KEY-----
2 MIIEowIBAAKCAQEA3dH7jNeEAMTR5oL7mh7v2Qk1jHvQXwknog1l1bXwGK/Ke
3 cvJusD3mHTdAIQbEveN3Yn1n+HaTd5cN18Tmwa3K1HtG1G1eWtWtVbVXNb+563
4 d43a1jtmr+QV03Ap23d5oWlqv+Thei8grnVC1vqkG6Ew1jnpof75KFphPmEoQ
5 Tbh398p/1s/hcOml+uwJWdjghnfq23KshCYG17D3Q/ST+lvjmfpgB8opRRZlum
6 GXAjGZm+J10rJKCB1K0A0CLVhoy4QVxUeUCX/h6QJTDFAEavbxcB88Ida1508
7 v6hJLOe3FXFWLGG13Cf9cYnPB8+VX8qH61TQIDmQhA0A1BDLV1P2uIXeC0im
8 8Q9eHf1C1TtEdmnaQcFBoWwS10D0z/tuwnP/0eYhLSP9AqYspBVT+9f4ePn
9 p3N11Aumog1B9MKCA5/e7JtmYRQdU6UeC6rGmL54y6tNE580m00j0phs3154
10 F9JLx40y56Hk4uWvNFnZb1A1V8y1Heo5dsvsCNS4HJZsnK7gJXPcxZumaU
11 /QpNsKtKdCZMo0zKDC21F1K4KPxU6amIT7Gwq089fC8XsK9gYQIUUKz+1G8
12 fvU9jX8QzD+5pA3x8Ghvm2cEbdOzwWesZtX7DAxoRMe1XQ99e4K1dVglaotGe3
13 I3uLTTsGyEABc6n9g9apYDIzCGKBAudH5qXP1/fmR82szv6B8H91m/3fNg8c
14 A622oy89EPc5/s3OYMTQndYJm1joAA6oFyujFHIOVxwqZT8ndB6m/mYehmA3
15 m8e1kE9mz48BVVH2h675+qBCKX4d8/LD78F7P8pxy5Y654mqGX788GyEA6tCb
16 jNs+Oa8Rkq1dxNLPh0DCNL7jaqNleB/8T7P6dK2ZKp71McysBpy1841Gox
17 wV979f7etVYtP1dubyeD0fPca444bJyqJ2M521L92Jr7QCE1B8P72KE1q
18 BCKX+UXG614qH5JyepPHCAwv0XtK712MbsCgYHExours2Z3K5QmQmAGKZ8
19 hg5U81n59g318C9Y5eHqZ0yUBdD9RA6+oIaccK2KCNj3d1F3V1KKu0a57B5
20 1K+2vA3Mpr386azVQEH15WYg1Ld05mfRnhUJ0a08v13P8IDLk92fK1/elwGAF
21 zAho8Om+n5NBc00HqRLKQ8GAL6P07U0BqJo0d0/er8LXJ76017OuJ8X5fPB2
22 /IqxmVha5eswos172hNIdtAqunGRBs2CkAUMJJuoz2TP8Twp/kn1eH5qN5n9a
23 TugQ7oh4utb6DefldZnvW7QyP2UQ6Jq22+8eOmJ3rasY0UJXHar8nScETH1aJ7K
24 2bLHAoGBAL4JHk17H76hcB+YaoV9FA2Y1z8Xyo+banKtXUu/Chy082FiRRRxp4
25 Bfn+MCKK05dCR8BVW/GjwBwz08Pxcv13V7kt7HsHgqJ8CZ19uE7Pevy2VKZG
26 AF9tP1dZqP2CwFKN0BL59P64JA801sv912Faa7pc+VRD0uq2

certs > MyRootCA_ca_cert.pem
1 -----BEGIN CERTIFICATE-----
2 MIIIDITCCangGAwIBAgIUOK7MsYctwshr9XaCUNhM8R0qkwdQV7KoZ1hvcNAQEL
3 BQAwDELMAKGA1UEBMCUExsDOAKBgWBAgMADQVDESMBAGA1UEBww5S0NsYm1h
4 YmFkMRwwGgYDVQKADAdGVuLUSVMSMRwDwyDVQDDAHNeV3yb3RDQTAEFwYNTEx
5 MTAxMzU0NDIafw0zMDExMDkxPzU0NDIaMFQxZjA3BjB8VBIAYTA1B1MQwGAgYDVQQT
6 DANJQ1QxZjA3BjB8VBIACMCU1zbGfFYWJhZDE0Q0A4A1UECgwHRkRVC1OVTERMAg
7 A1UEAwITX150290QeWggle1PMKCSqGSIb3DQEBAQUAAIIBBgKGAQ1BAQDD
8 0F0H1QAMFRGsmwHu/M6qTHO089CT6e1CTUx74aAT8P5y4H6Wp4eJ0n0h5S
9 943d1KeX4dpR31w21FD0B8ro1FOCIYh6h2hZP59Redv6znd3Jd6d01aus18qJc1
10 nd1I7CmQ/SM161SCudGLMS060bTMD0mh7t10mef+YShAhsCn1H2nKz+FuSVY
11 67A1Z20Gcd+pncqyEJgbXsH1D93P6K+0YmngABG1JF1LSV2ZcCk2b06U6smQL2M
12 Tr041tU2jLh8j57W4HhA1M8BARqVHE3tTw1qX0Nty/gHcs57d4RcXASVYj
13 dxP1x1c/zT5VvcGqHw1AgMBAAEgJuzBRMA8GA1UdEwEB/wQFMAMBAF8wQYDVRR80
14 BBVEFDd3M2GjHA13un5/5q5s3BcTrINBMB8GA1Ud1wQYBAFAD3M2GjHA13un5/
15 5q5s3BcTrINBMA8GCSqGSIb3DQEBCCUAA1BAQAcbrcMhJ1/nbLVOK7qTJRtNZDS
16 n5JceHaeqsk0Mzq190KIHP2PmkY1VqMwS8B6GZHS1/Phx3+1DXayC11gk8m
17 hBfBfBgG5NmvITX314w6oyGdWfmg5qyH4598R0ob0a1g0uNRVSYldpCR
18 UGKc1d6J/77h3ajkc11kg2q4w52upMee590g522JKR/z4P9Y5RGPd1JCvYe
19 a4/Lq36Fwz1MLPrLPW+uv2FmFKewGmF1HEX3JhTz84SEPLhufNmMEYqxu
20 xu6t85gmE951FctbCCLeyh0gyp2rqHVd144csqY1zMG1dcYXQ2w90ff
21 -----END CERTIFICATE-----
```

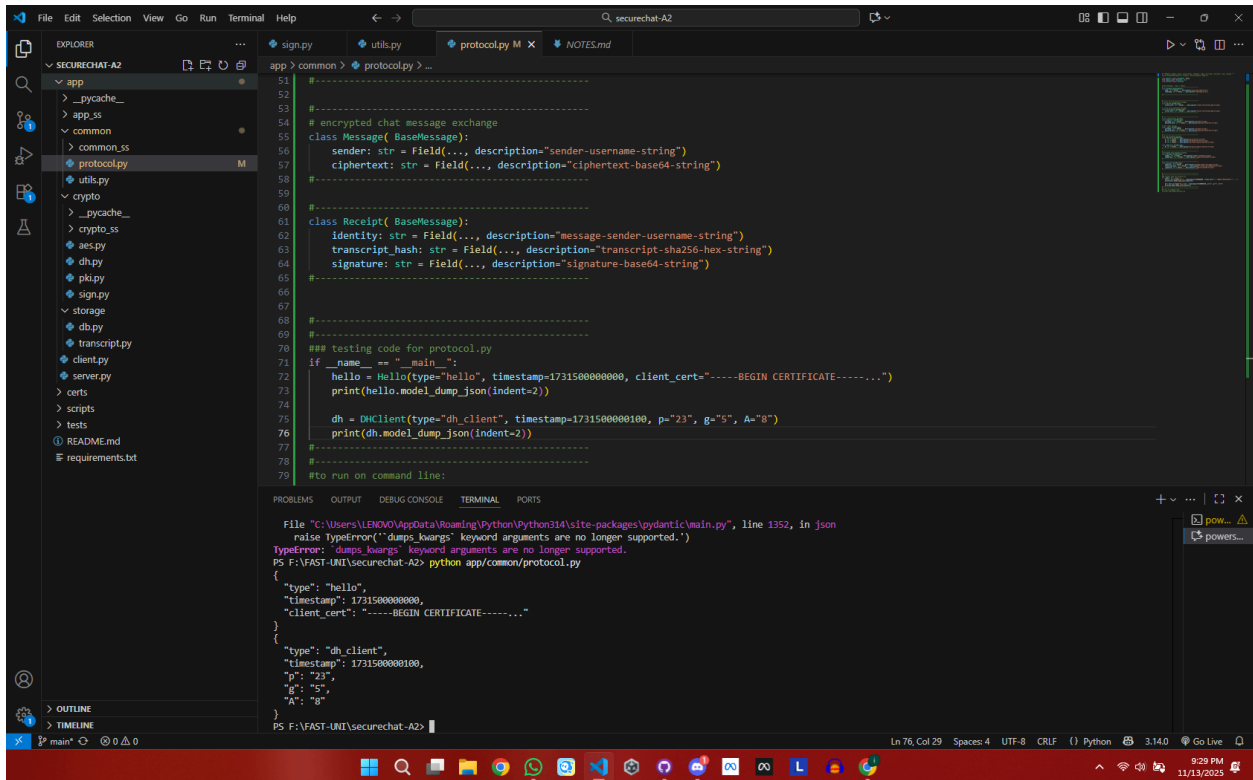
Table of file properties:

Mode	LastWriteTime	Length	Name
d----	11/10/2025 5:10 PM		app
d----	11/10/2025 5:10 PM		scripts
d----	11/10/2025 5:10 PM		tests
-a----	11/10/2025 5:10 PM	4715	README.md
-a----	11/10/2025 5:10 PM	54	requirements.txt

Terminal output (continued):

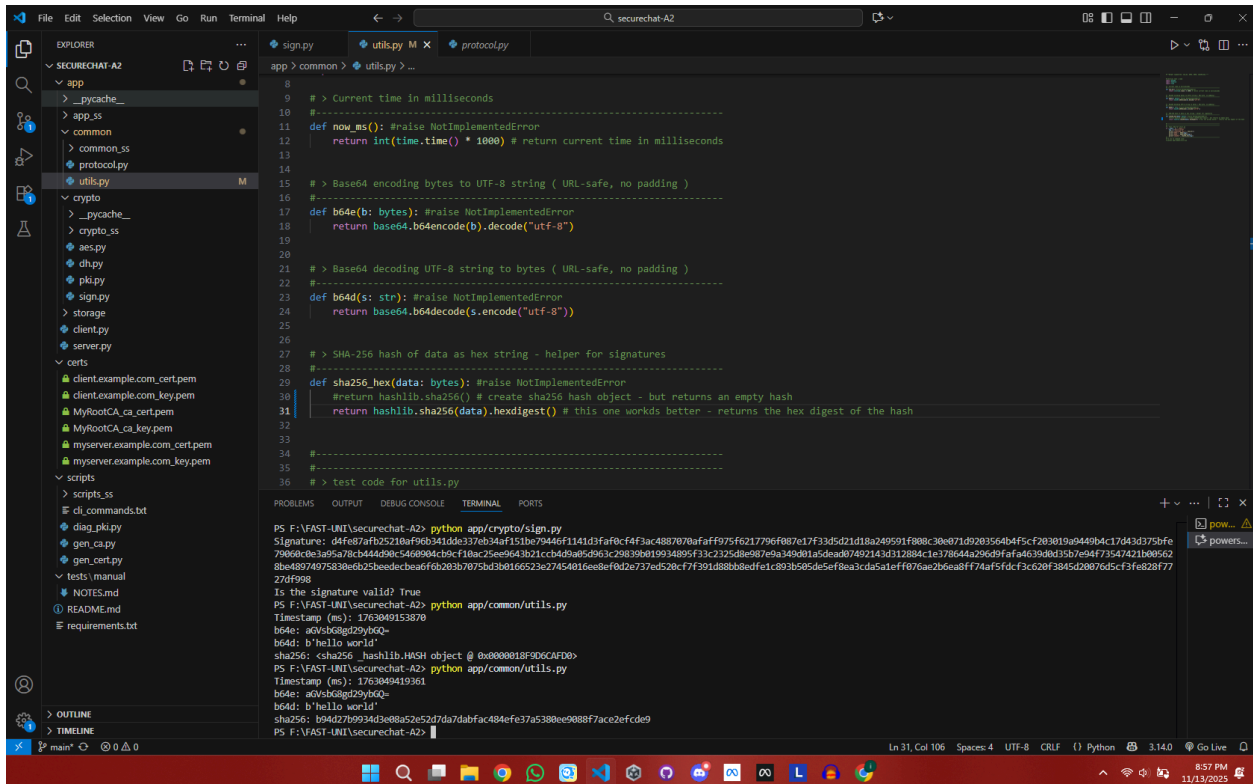
```
PS F:\FAST-UNIT\securechat-A2> python scripts/gen_ca.py --ca-name MyRootCA --output-dir certs
F:\FAST-UNIT\securechat-A2\scripts\gen_ca.py:45: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to
represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
.not_valid_before(datetime.datetime.utcnow())
F:\FAST-UNIT\securechat-A2\scripts\gen_ca.py:46: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to
represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
.not_valid_after(datetime.datetime.utcnow() + timedelta(days=1825)) # ~5 years
Root CA generated successfully!
Private Key: certs\MyRootCA_ca_key.pem
Certificate: certs\MyRootCA_ca_cert.pem
PS F:\FAST-UNIT\securechat-A2>
```

3.5 Common Functionality Tests



```
File Edit Selection View Go Run Terminal Help
securechat-A2
EXPLORER
  app
  _pycache_
  app.py
  common
    protocol.py
    utils.py
  crypto
  _pycache_
  crypto.py
  aes.py
  dh.py
  pk.py
  sign.py
  storage
  db.py
  transcript.py
  client.py
  server.py
  certs
  scripts
  tests
  README.md
  requirements.txt

protocol.py
51 #-----
52 #-----
53 # encrypted chat message exchange
54 class Message(BaseMessage):
55     sender: str = Field(..., description="sender-username-string")
56     ciphertext: str = Field(..., description="ciphertext-base64-string")
57 #-----
58 #-----
59 #-----
60 class Receipt(BaseMessage):
61     identity: str = Field(..., description="message-sender-username-string")
62     transcript_hash: str = Field(..., description="transcript-sha256-hex-string")
63     signature: str = Field(..., description="signature-base64-string")
64 #-----
65 #-----
66 #-----
67 #-----
68 #-----
69 #-----
70 ## testing code for protocol.py
71 if __name__ == "__main__":
72     hello = Hello(type="hello", timestamp=1731500000000, client_cert="-----BEGIN CERTIFICATE-----")
73     print(hello.model_dump_json(indent=2))
74
75     dh = DHClient(type="dh_client", timestamp=1731500000100, p="23", g="5", A="8")
76     print(dh.model_dump_json(indent=2))
77 #-----
78 #-----
79 #to run on command line:
80
81 File "C:\Users\LENOVO\AppData\Local\Programs\Python\Python314\packages\pydantic\main.py", line 1352, in json
82     raise TypeError(f"dumps kwargs: {kwargs} keyword arguments are no longer supported.")
83 TypeError: dumps kwargs: keyword arguments are no longer supported.
84 PS F:\FAST-UI\securechat-A2> python app/common/protocol.py
85 {
86   "type": "hello",
87   "timestamp": 1731500000000,
88   "client_cert": "-----BEGIN CERTIFICATE-----"
89 }
90 {
91   "type": "dh_client",
92   "timestamp": 1731500000100,
93   "p": "23",
94   "g": "5",
95   "A": "8"
96 }
```



```
File Edit Selection View Go Run Terminal Help
securechat-A2
EXPLORER
  app
  _pycache_
  app.py
  common
    protocol.py
    utils.py
  crypto
  _pycache_
  crypto.py
  aes.py
  dh.py
  pk.py
  sign.py
  storage
  db.py
  transcript.py
  client.py
  server.py
  certs
  scripts
  tests
  README.md
  requirements.txt

utils.py
9 # > Current time in milliseconds
10 #-----
11 def now_ms(): #raise NotImplementedError
12     return int(time.time() * 1000) # return current time in milliseconds
13 #-----
14 #-----
15 # > Base64 encoding bytes to UTF-8 string ( URL-safe, no padding )
16 #-----
17 def b64e(b: bytes): #raise NotImplementedError
18     return base64.b64encode(b).decode("utf-8")
19 #-----
20 #-----
21 # > Base64 decoding UTF-8 string to bytes ( URL-safe, no padding )
22 #-----
23 def b64d(s: str): #raise NotImplementedError
24     return base64.b64decode(s.encode("utf-8"))
25 #-----
26 #-----
27 # > SHA-256 hash of data as hex string - helper for signatures
28 #-----
29 def sha256_hex(data: bytes): #raise NotImplementedError
30     #return hashlib.sha256(data).hexdigest() # this one works better - returns the hex digest of the hash
31     return hashlib.sha256(data).hexdigest()
32 #-----
33 #-----
34 #-----
35 #-----
36 # > test code for utils.py
37
38 PS F:\FAST-UI\securechat-A2> python app/crypto/sign.py
39 Signature: d4f07f7b2c210af96b341d4e3370b34af151be79446f1141d3faf0cf4f3ac4887070aff975f6217796f087e17f33d5d21d18a249591f808c30e871d9203564b4f5c2030190440b4c174343752bf
79060e3a95a70cb44d90c546804cbcf18ac25ee9643b21c4b9a95d963c298390b19934895f33c2325d8e087e9a349d01a5dead07492143d312884c1e378644a296d9fafa4639d0d35b7e94f73547421b08562
8be48974975830e625beedecbea6f6b203b7075b3b0166523e27454016ee8e8f0d2e737ed520cf7f391d88b8edf1c893b505de5ef8a3c4d5a1eff076ae2b6ea8ff74af5dc3c20f3845d20076d5c3f8e28f77
27d4998
40 Is the signature valid? True
41 PS F:\FAST-UI\securechat-A2> python app/common/utils.py
42 Timestamp (ms): 1763049153870
43 b64e: aQvSbGgI29ybGQ=
44 b64d: b'hello world'
45 sha256: cba256 _hashlib.HASH object @ 0x0000018f906CAFDE
46 PS F:\FAST-UI\securechat-A2> python app/common/utils.py
47 Timestamp (ms): 1763049419361
48 b64e: aQvSbGgI29ybGQ=
49 b64d: b'hello world'
50 sha256: b04d709294d3e88a52652d7da7dabfac484fe37a5380ee988f7ace2efcde9
51 PS F:\FAST-UI\securechat-A2>
```